each.

What do these various declarations do? Basically, they add navigation capabilities to the model objects. Because Rails added the `belongs_to` declaration to `LineItem`, we can now retrieve its `Product` and display the book's title:

```
li = LineItem.find(...)
puts "This line item is for #{li.product.title}"
```

To be able to traverse these relationships in both directions, we need to add some declarations to our model files that specify their inverse relations.

Open the `cart.rb` file in `app/models`, and add a call to `has_many`:

rails7/depot_f/app/models/cart.rb

```
class Cart < ApplicationRecord
»   has_many :line_items, dependent: :destroy
end
```

That `has_many :line_items` part of the directive is fairly self-explanatory: a cart (potentially) has many associated line items. These are linked to the cart because each line item contains a reference to its cart's ID. The `dependent: :destroy` part indicates that the existence of line items is dependent on the existence of the cart. If we destroy a cart, deleting it from the database, we want Rails also to destroy any line items that are associated with that cart.

Now that the `Cart` is declared to have many line items, we can reference them (as a collection) from a cart object:

```
cart = Cart.find(...)
puts "This cart has #{cart.line_items.count} line items"
```

Now, for completeness, we should add a `has_many` directive to our `Product` model. After all, if we have lots of carts, each product might have many line items referencing it. This time, we make use of validation code to prevent the removal of products that are referenced by line items:

rails7/depot_f/app/models/product.rb